

# **Designing innovative products -introduction-**

---



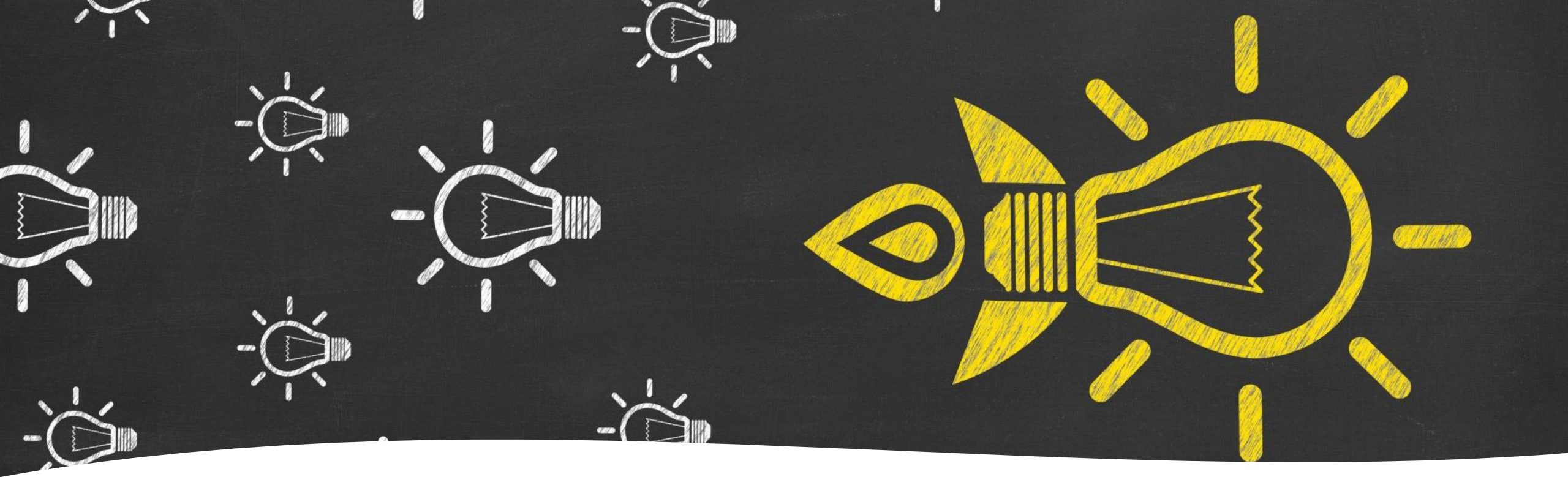
# Let's get organized

- Final grade =
  - 40% milestones +
  - 60% demo Day
- Each 2 week of delay = **-2p** from the milestone grade.
  - Each milestone is delivered in your team's GitHub repository
  - Teams should participate to the meets with the challenge owner.
- All team members should participate to the DemoDay (week 14).
- Max 2 absences from the lab

MS Teams code: **3r18fhj**

Course site:

[ilenitudor.github.io/ppi\\_ubb](https://ilenitudor.github.io/ppi_ubb)



**What is  
innovation?**



# 1969

Humans walk on the Moon



# 1970

Someone puts wheels on a suitcase

The wheel is ~5,500 years old.  
Why did it take so long?

# Innovation Is not **Invention**

---

- May include **invention** but requires many other things:
  - Deep understanding of customers
  - How to work with partners
  - How it will pay for itself
- Innovations are based on **previous** advances. They don't have to be new to the world.



KEEP  
CALM  
AND  
COMBINE

# Who invented it? Who won?

## INVENTED IT

### **Kodak**

first digital camera (1975)



## WON

### **Everyone else**

Kodak went bankrupt in 2012

### **Xerox PARC**

the graphical desktop, Alto (1973)



### **Apple**

the Macintosh (1984)

### **Google**

the Transformer (2017)



### **OpenAI**

ChatGPT (2022) — the “T” in GPT

*“Research turns money into knowledge. Innovation turns knowledge into money.”*

— attributed to Geoffrey Nicholson, 3M

Def.

Innovation requires  
**Identifying<sup>1</sup>** the problems  
that **Matter<sup>2</sup>** and **Moving  
through Them<sup>3</sup>**  
Systematically to **Deliver  
Elegant Solutions<sup>4</sup>**.

1

## Identifying the problems

Challenge owners · business labs

2

## ...that Matter

Your innovation hypothesis: is it worth solving?

3

## Moving through them systematically

Agile (today) · Specify → Build with AI → Verify

4

## Deliver elegant solutions

UX (Lab 2) · AI (Labs 3–6) · Pitch (Lab 7)



Innovation is a team sport, it's not the domain of rare genius or chosen few. Anyone can (and should) innovate, and there is no longer an excuse

---



2026: the invention is done — the models exist, building got cheap.  
The hard part now: knowing WHAT is worth building.

# Lab 1

How teams build products



# Project Management Horror Stories



"Have you ever been on a team project where the requirements changed at the last minute?"



"Have you ever submitted a project, only to be told it wasn't what the 'client' (teacher) wanted at all?"

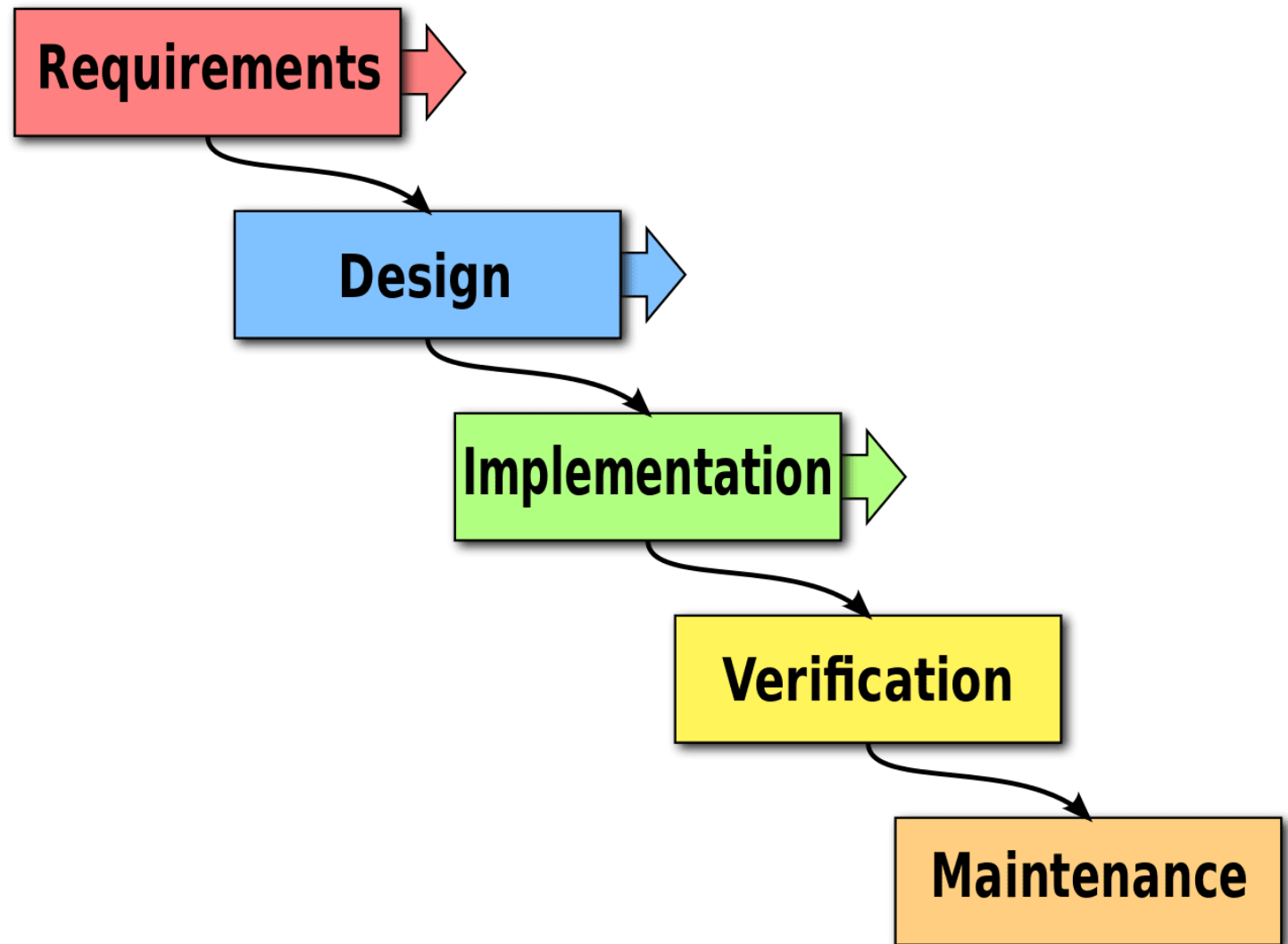


"Have you ever been stuck waiting for one person to finish their part so you could start yours?"

# Waterfall Model Project Management

---

- **Sequential:** each phase ends before the next one starts.
- Changing your mind late is **expensive**.
- Still the most used: **44%** of project professionals use it always/often (58% in 2020).
- Rising: **hybrid**, from 20% to 32%.



# Agile: small steps, working product

- **Small cycles**

Sprints of 1–4 weeks. Plan, build, test, review.

- **A working product after every sprint**

Not a pile of parts.

- **Feedback changes the plan**

The customer sees it early and often.

NOT LIKE THIS



LIKE THIS



Every step is something the user can already use, and tell you what is wrong with it.

# Agile vs Waterfall

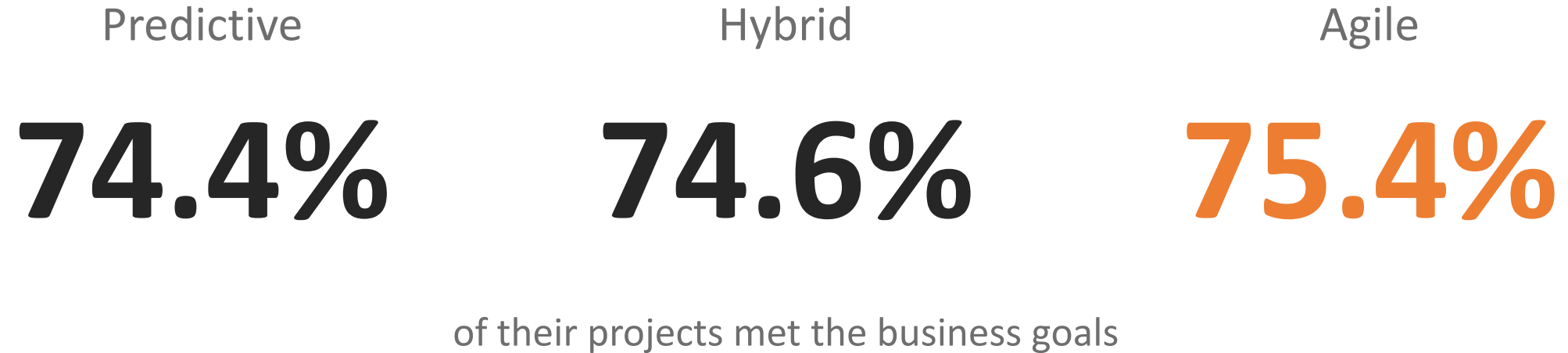
|                   | Waterfall                | Agile               |
|-------------------|--------------------------|---------------------|
| Planning          | Whole project, upfront   | 1–4 weeks at a time |
| Customer involved | At the start and the end | Every sprint        |
| Change            | Expensive, avoided       | Expected            |
| Working product   | At the end               | After every sprint  |
| Team              | Hierarchy                | Self-organizing     |
| Best when         | Requirements are fixed   | Uncertainty is high |

## Waterfall still wins when change is expensive or regulated

Construction: 76% use predictive approaches always/often.

Avionics (DO-178C), car safety (ISO 26262), medical software (IEC 62304).

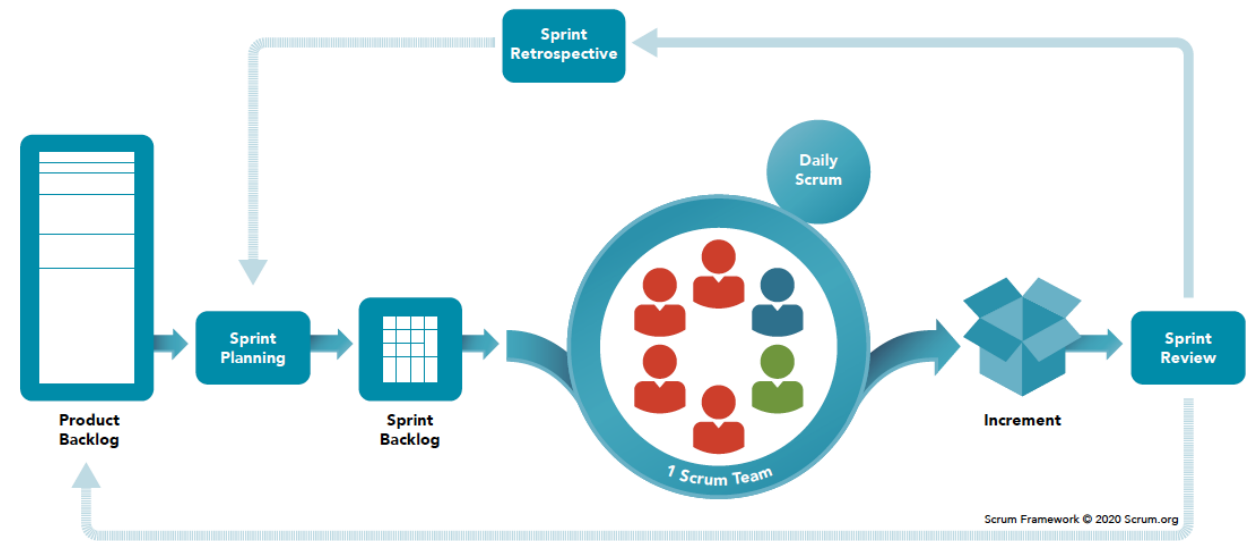
# Plot twist: which approach succeeds most often?



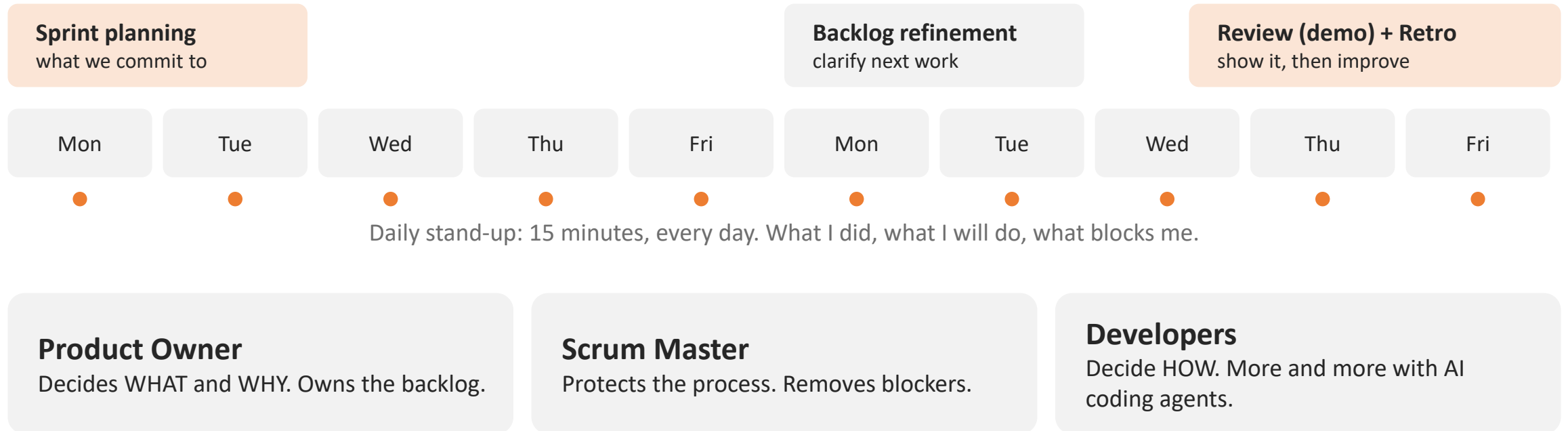
Almost identical. The approach alone does not make projects succeed.  
**The team does.**

# Scrum

- A **Product Owner** orders the work for a complex problem into a Product Backlog.
- The **Scrum Team** turns a selection of the work into an Increment of value during a Sprint.
- The **Scrum Team and its stakeholders** inspect the results and adjust for the next Sprint.
- **Repeat**



# Agile in a company: a typical 2-week sprint



Reality check: most companies mix both.

*"Like being agile on the inside and predictive on the outside."*

— Felipe Garro, Indorama Ventures, in PMI Pulse of the Profession 2024

Typical tools: Jira · Confluence · GitHub · Slack / Teams

# Kanban: it started in a supermarket

1950s: Taiichi Ohno (Toyota) studies American supermarkets.

Shelves are refilled only when customers take products.

**Kanban (“signboard” in Japanese): work is pulled, not pushed.**

1

## Make work visible

Every task is a card on the board.

2

## Limit work in progress

Each active column has a max, e.g. Doing (4), Review (2).

3

## Pull, don't push

Take a new card only when the column is under its limit.



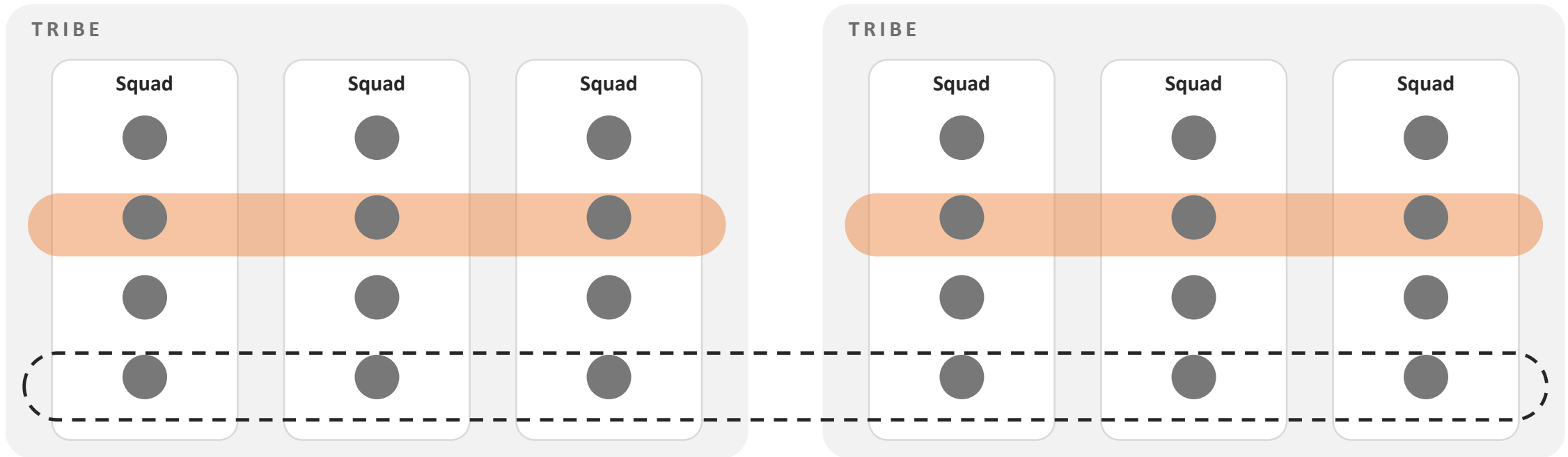
**Stop starting.  
Start finishing.**



# Real-World Agile: How Spotify Builds a Product for 777 Million Users

How do you innovate and release features constantly without creating chaos?

# Squads, Tribes, Chapters, Guilds



## Squad

One long-term mission. E.g. Android app, Radio, Payments.

## Tribe

Squads in related areas, < 100 people. E.g. Music player, Backend infrastructure.

## Chapter

Same skill, same tribe. E.g. testers, web developers, backend.

## Guild

Anyone interested, across tribes. E.g. Web technology, Testing, Agile coaching.

# Plot twist: Spotify never fully used the “Spotify model”

---

*“Even at the time we wrote it, we weren’t doing it. It was part ambition, part approximation.”*

— Joakim Sundén, agile coach at Spotify 2011–2017

- Meanwhile, the world copied it. ING bank (2015): ~350 squads in 13 tribes.
- What went wrong at Spotify: lots of autonomy, little alignment; nobody clearly owned delivery.

**Lesson**  
**Copy the principles, not the org chart.**

A Spotify engineer walks into a bar.

The bartender asks 'What'll you have?'

The engineer says:

'I'm not sure, but can you just play me a few **random drinks** and I'll tell you which one I like?'



# XP: 1999 ideas, 2026 superpowers

**Kent Beck's idea (1990s): if a practice is good, turn it up to the max.**

Code review is good → review all the time: pair programming.

Testing is good → write the test first: TDD.

Integration is good → integrate many times a day: CI.

**That's the “extreme” in Extreme Programming.**

| XP practice             | With AI, in 2026                                                        |
|-------------------------|-------------------------------------------------------------------------|
| Pair programming        | You + an AI assistant. “Your AI pair programmer” (GitHub Copilot, 2021) |
| Test-driven development | Tests are the spec the AI’s code must pass                              |
| Continuous integration  | Every AI change is checked automatically                                |
| Refactoring             | Cleaning up the technical debt AI leaves behind                         |
| Small releases          | Small changes you can actually review                                   |

In 2026 everyone has a fast junior developer: AI. XP’s habits are how you keep it in check.

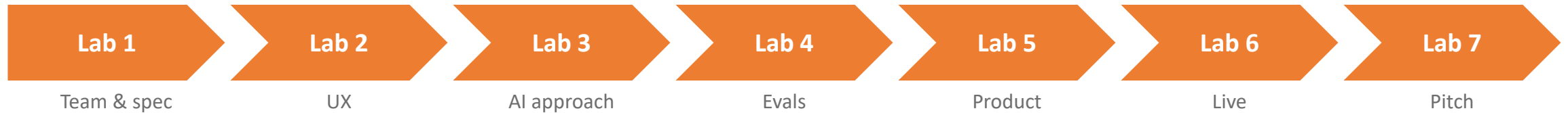
## Pair Programming 101

---

The art of two people making the same typo at the same time.



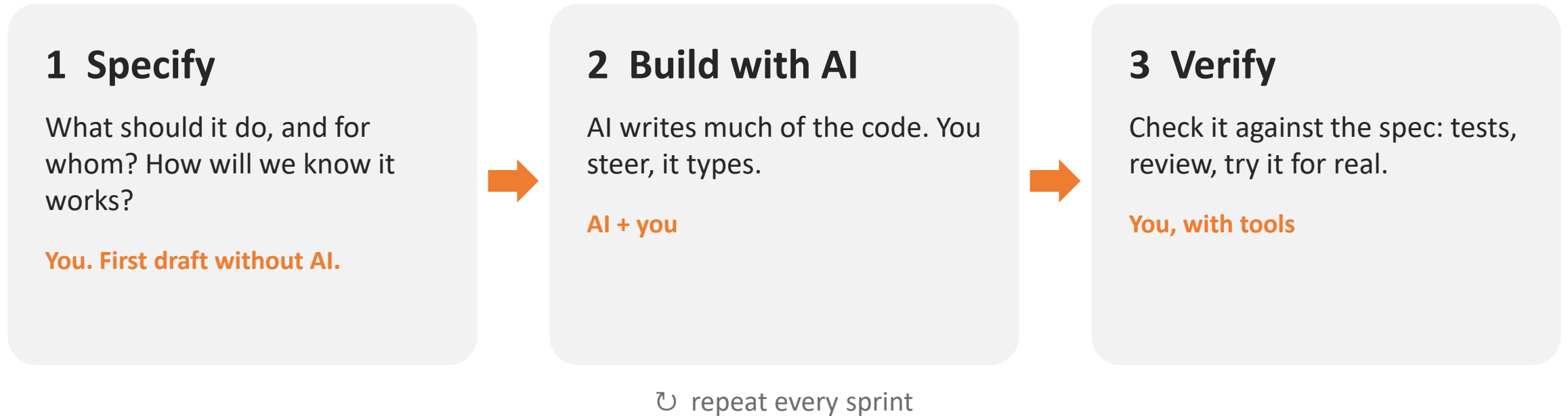
# How we work in PPI: 7 sprints → Demo Day



**Every tech lab:** first the ideas and tools, then one hour of project discussions.

| In a company             | PPI                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------|
| Product Owner            | Your challenge owner                                                                              |
| Sprint (2 weeks)         | The 2 weeks between your tech labs                                                                |
| Sprint review + planning | Every tech lab: one hour of project discussions with me. Show what works, agree on the next step. |
| Daily stand-up           | A short update in your team chat, 2–3 times a week                                                |
| Board                    | GitHub Projects: To do · Doing · Done                                                             |
| Retrospective            | In the project discussion: what to keep, what to change                                           |

# Inside every sprint



**Done = verified against the spec.**

*Not “it runs on my laptop”.*

# Before next time

☐

## **Read the challenges**

Before the meeting. Mark the ones you would like to work on.

☐

## **Meet the challenge owners**

Tuesday, 6 October, 8:00.

☐

## **Think about your team**

Who do you want to work with? Include your MIRPR colleagues if you share a challenge.

☐

## **Prepare a few questions**

Optional: for the challenge owners. Ask about the user, the problem, the data.

# Milestone 1: Spec

(due in 2 weeks · RO: week 3 · EN: week 4)

**Theme:** Know what you are building before AI builds it.

## Tasks:

- **GitHub project:** a repository with SPECS.md. It can be the same repository as your MIRPR colleagues'.
- **Specification list (SPECS.md):** team & challenge, the problem and the user, your innovation hypothesis, and 3–5 user stories, each with a “Done when”.
  - As a dentistry student, I want to practise with a simulated patient, so that I learn to ask the right questions.  
Done when: the simulated patient answers 20 test questions consistently with the case file.

**Rule:** write the first version **yourselves**, without AI. Then ask an AI to find the gaps.

**Deliverable:** SPECS.md in the root of your team repository on GitHub. No MS Teams upload: I check the repository at the deadline.

A decorative graphic on the left side of the slide features a red pushpin stuck into a map. The map shows streets and landmarks, with the pushpin pointing to a specific location. The graphic is partially cut off by the left edge of the slide.

# Bibliography

---

- [PMI, Pulse of the Profession 2024](#)
- [The Scrum Guide](#)
- [H. Kniberg, Making sense of MVP \(2016\)](#)
- [H. Kniberg, A. Ivarsson, Scaling Agile @ Spotify \(2012\)](#)
- [J. Lee, Spotify's Failed #SquadGoals \(2020\)](#)
- [McKinsey, ING's agile transformation \(2017\)](#)
- [Kanbanize, What is Kanban?](#)
- [Chrysler Comprehensive Compensation System](#)
- [NUS-Google, Reshaping Undergraduate CS Education in the Generative AI Era \(2026\)](#)
- [Waterfall diagram: monday.com](#)